Team Project Report - Phase 1 Microservice 1 Checkpoint

Note: We strongly recommend reading through both checkpoint and final report template before starting the project!

Phase 1, Microservice 1 Checkpoint summary

Team name: dwendwen
Members (names and Andrew IDs):

Wangshu Li: wangshul
Qing Deng: qingd
Jiarui Zhang: jiaruiz

Please color your responses as red.

Overview:

In Phase 1, our objective is to make sure all the teams acquire the following skills through various tasks:

1. Microservice 1 and Microservice 2

Learning Objective	**How it will be graded**
Comparing web frameworks	Test submissions, report, throughput
Comparing different types of load balancers	Test submissions, report, throughput
Exploring instance types and number, and architecture	Report, throughput
Web-tier resilience: Malformed requests	Test submissions, manual grading
Web-tier deployment orchestration using Kubernetes, Helm and Terraform	Report, manual grading
Profile and optimize computationally expensive tasks	Report, throughput

2. Microservice 3

Learning Objective	**How it will be graded**

Break down requirements and **design functional database schemas** based on a given dataset	Test submissions, report, throughput
Design and implement effective **ETL pipeline** given the desired output	Budget usage, report
Implement **TDD** by writing unit tests on small data subsets	Report, manual grading
Storage-tier **deployment orchestration** using Kubernetes, Helm and Terraform	Report, manual grading
Develop **character encoding-aware** solutions	Test submissions
Recognize **skewness** in the Twitter dataset and implement how to address it/remove it	Test submissions
Identify **MySQL/HBase APIs** to use and integrate the APIs into the web service	Test submissions, throughput, manual grading

3. Peripheral skills

Learning Objective	**How it will be graded**
Using git effectively	Report, git statistics
Carrying out code reviews regularly	Report
Efficient task partition to enable parallel development and testing	Report
Timely and effective communication with the team	Report
Effective monitoring resources during the live test	Report
Clear concise documentation through reports	Report, manual grading

Phase 1 Checkpoint 1 Rubric

- Each unanswered bullet point = -4%
- Each unsatisfactory answer = -2%
- Use the report as a record of your progress, and then condense it before submitting it.
- Always use your own words (paraphrase); we will check for plagiarism.

Question 0: Knowing each other

After your first meeting, each member is required to submit the following Google Form before the Microservice 1 Checkpoint Report submission deadline:
https://forms.gle/Co9wnFHSWMuPsVXw6

Question 1: Scope of the project

You may need to read the write-up several times to get a general understanding of tasks. Discuss the following questions as a team:

- What skills you have learned in previous projects will come into play here?
  The use of AWS, Azure, GCP and their API.
  Writing Dockerfile.
  Use kubernetes for deployment and service, including load balancers, pods, ingress, nodePort.
  Use helm for kubernetes package management.
  Use Mysql for sql database.
  Use hbase for no-sql database.
- Have you used any web framework before? How will you compare the different web frameworks and choose one suitable for this phase? (Answer in less than 100 words)
  Our framework choices include Python with Django/FastAPI, Java with Spring Boot, Jooby and Vert.x, and C# with ASP.NET Core. Each implementation keeps the same request and response contract. We compare correctness, RPS, latency and CPU utilization using consistent workloads and resource limits. ASP.NET Core Minimal APIs keep routing separate from service logic, while its asynchronous database APIs support the I/O-oriented work in m3.
- We want you to learn the cloud-native development and deployment approach. Explain the differences between a monolithic application and having multiple microservices.
  A monolithic application can be disassembled into a set of independent services that are developed, deployed and maintained separately. Microservice architecture is more fault-tolerant because it's resistant to single-point-failure, a microservice can work normally with other microservices crash.
  Also, monolithic applications are more difficult to scale, while we can easily scaling up or scaling out for microservices, which makes our project more resilient.
- The project requires you to work with either MySQL or HBase. List at least three fundamental differences between these two database systems.
  1. Mysql manages structured data and it's relational. Hbase manages both structured and unstructured data and it's non-relational.

2. Mysql is supported by many programming languages but Hbase is only supported by Java
3. Mysql follows the ACID property. Hbase follows the CAP therom.

- Use one or two short sentences to explain how the index in MySQL works.
  MYSQL indexes are stored in B-trees. If the table has an index for the columns in the query, MYSQL can quickly determine the position to seek in the middle of the data file without having to look at all the data.
- Use one or two short sentences to explain how HBase finds a row using its data structure.
  Hbase locates specific data first by row key, then by version and timestamp.

Question 2: Team

As mentioned on Piazza, team formation is critical for the success of the team.

- What criteria did you use to choose team members, and when did you begin that process? For each bullet point, please explain in 1 to 2 sentences:
  - Did each member simply accept an invitation to join this team?
    Yes
  - Did members observe the performance and behavior of other team members? If so, what traits did you observe?
    Yes. We observed which tasks each member performed and how often each one committed his/her work.
  - Did your team follow the recommendations provided by the course staff?
    Yes. One member is good at ETL; one is skilled at web frameworks; one is fluent with databases.
  - Did you form the team based on friendship?
    Partially based on friendship, but more focused on the skills each one has.
  - Did you follow the questionnaire posted on Piazza to find your teammates?
    We followed the guideline on Piazza to search for teammates.

Regular meetings are necessary to review status updates, to integrate code and to discuss solutions to possible challenges.

- When will your team hold regular team meetings?
  Primarily on evenings and weekends.
- Are you using a common calendar to notify members about the meetings?
  Yes, we are using Google calendar.
- Is there a set of action items for each member to work on after the meeting?
  Yes.
- Which project management tool will you use to keep track of these tasks? (some options: Asana, Trello)
  We use Asana to assign tasks to each member.

The team members should have met several times by now. You might have different skill sets and interests in working on various parts of the project. Tell us more about yourselves and your plan of collaboration by answering the following questions.

- Describe the skill sets of each of your team members.
  Qing Deng is good at Python and ETL. Jiarui Zhang is skilled at web frameworks and applications; Wangshu Li is fluent with Java and databases.
- Describe the interests of each of your team members in the project tasks. (we encourage you to go outside your comfort zone and work on tasks that might challenge you and give you the opportunity to learn new skills)
  Qing Deng is interested in ETL. Jiarui Zhang is interested in building web applications. Wangshu Li is interested in deployment and databases.
- Which collaboration platform do you plan to use for communication (e.g., Asana, Slack, Trello, etc.)? Why did you choose this platform? How do you plan to use this platform?
  We plan to use Asana to manage tasks because it's easy to use. We plan to use Wechat for daily communication because everyone uses it frequently.

The team project is known to require a heavy workload. The division of labor has to be done in a careful manner to make sure that everyone is contributing to the progress of the team.

- How do you plan to divide the work and why?
  For example, in m1, Qing Deng wrote the encoding and decoding codes; Jiarui Zhang built the service with web frameworks; Wangshu Li deployed the service and tuned the pamameters like instance type and number of nodes. We tried to focus on different parts so that we can work concurrently.
- Which part will each of you work on?
  For future tasks, Qing Deng will focus on ETL. Jiarui Zhang will focus on web applications. Wangshu Li will work on databases.

Question 3: Design and exploration

During the implementation of the team project, you have the chance to explore different ETL tools, web frameworks, database schema designs as well as Kubernetes and EMR cluster configurations. Show us your plan of exploration and how it will impact your design decisions. You don't necessarily need to implement them or apply them to your actual implementation.

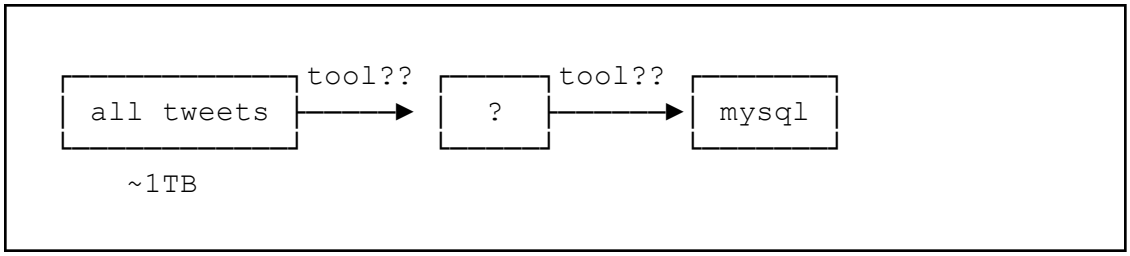
- Which programming language / web framework combinations did you or will explore? Name two combinations that you are considering exploring. Use one or two sentences to explain the underlying implementation of each of them (e.g. Compiled language? Event-loop? Multi-Thread? Other?). Note that the combinations you choose to explore can be in the same programming language.
  Django and Spring Boot provide familiar application structures. Vert.x uses event loops, while Jooby builds on NIO servers. C# with ASP.NET Core compiles to .NET intermediate language and uses Kestrel for HTTP requests. The m1 implementation separates routing from QR matrix construction and Logistic Map encoding.

CPU-intensive encoding runs in the request handler; database operations in m3 use asynchronous I/O and pooled connections.

- What parameters are you going to use to compare the different web frameworks? Provide at least 3 parameters.

  Throughput, latency, correctness, CPU utilization and memory usage.
- What are the differences between Application and Network Load Balancers? Describe one other technique that achieves load balancing and request routing. How are you going to compare them in order to decide which one is suitable for your web service?
  1. Network load balancer works at ISO layer 4, TCP and UDP, whild Application Load Balancer works at layer 7, HTTP. ALB can send static response without backend, whild NLB isn't able to, which could help us test the load balancer and routes in hands with the development of application.
  2. Ingress
  3. We are going to first look up the documentation that comment on the difference betweent these services and figure out which fits our situation the best.
- What's your preliminary plan for the ETL process? Can you express the process as a directed graph (see below)? What measure are you going to take to ensure the correctness of the ETL result? What can you do to enable reuse if you had to rerun the ETL process?

  Example Directed Graph (you may illustrate your idea in any way you like):



  We divided the ETL jobs into several steps. Extracting data from data source, clean the data, and finally load the data into different tables/schemas. To make sure the correctness, we need to conduct validation after each transformation to prevent failure cascade. The mesasures could be the number of total rows. We should decouple each adjacent step to improve re-usability.
- Which ETL tools (Hadoop Streaming, Hadoop Batch Processing, Apache Spark, etc.) do you plan to use? How does it fit into your ETL process? What about its development convenience, expressiveness, runtime performance and tools for debugging given the large dataset?

  We plan to use Apache Spark since later the individual project will also include this topic, the learning cost would be lower than other ETL tools.
- In order to return the response, you need to store the processed data in MySQL or HBase for Microservice 3. Which data is needed? How do you plan to design the database schema? What are some considerations behind your schema design?

m3 needs contact IDs, tweet text, hashtags, scores and user profiles. MySQL can store precomputed contact records keyed by user ID; the Java and C# services read this serving data. HBase is a row-key-oriented alternative.

- Provide the preliminary architectural end-to-end design of your microservices for this project, including your Kubernetes cluster, load balancer, web-tier, database, etc. Discuss how the computational resource of your Kubernetes will be allocated to which part of your web service? How would you distribute the load across different machines? (You can provide an illustration of your architecture and of your cluster design here)

  A shared HTTP entry point routes /qrcode, /blockchain and /twitter to independent service Deployments. Java uses Jooby/Vert.x; C# uses ASP.NET Core. MySQL remains separate. AWS provisioning uses kOps; the Azure path uses Azure DevOps to build images in ACR and release them to AKS with Helm. Kubernetes distributes replicas across worker nodes, with CPU and memory limits controlling each service.

- Describe at least two methods to provision a self-managed Kubernetes cluster. Compare the methods and list their advantages and disadvantages.

  KOps and manually deploy on Cloud Provider's Console.

  The advantages and disadvantages of KOps is that it takes time to write these configurations files, which could be an overhead. However, the flip-side is that it is easy to destroy all resources.

  The advantages and disadvantages of console is that though it is interactive and straight for deploying resources, it could be a headache to destroy due to the resource dependency complexity.

- Since one pod in your Kubernetes cluster is not enough to reach the target throughput, how do you plan to scale up your web service? Explain which method you will adopt and how you reached this decision.

  We are going to use Horizontal Pod AutoScaling, which scale in and out based on various metrics.

- Your team will be required to orchestrate the deployment of your microservices using Terraform, Kubernetes and Helm. Briefly explain the functional role of these components.

  Terraform describes infrastructure resources; Kubernetes schedules containers and maintains Deployments and Services; Helm packages their manifests. Azure DevOps connects validation, container builds and Helm releases in a pipeline, with ACR holding the images and AKS running the workloads.

- Reading the Phase 1 report template will provide your team with many hints on how to successfully complete this project and not waste many hours. What hints were useful to your team after reading the report template?

  We found that the choice of framework is vital and indeed spent more attention to comparing different frameworks to achieve between performance.

Question 4: Time

After getting a big picture by answering the above questions, you may start thinking how to distribute the workload among you and your teammate. Conducting sufficient exploring and optimization will not only help you achieve target RPS in Phase 1 but also greatly expedite your process in Phase 2 and 3. The following questions provide a guideline for you and your teammate to plan your time wisely.

- What tasks do you plan to divide phase 1 into? Show us the internal team deadlines you have decided for each task. (provide a timeline as a table with tasks, who will work on which tasks by which days/weeks)
  Currently, we have finished m1.
  Jiarui will focus on m2 logic codes and will try to finish by March 6.
  Wangshu will focus on m3 logic codes and will try to finish by March 20.
  Qing will focus on ETL, perhaps with the help of the other two members since ETL is time-consuming.
- ETL will be very time-consuming. It's very common that you have to rerun the ETL job because of a trivial error in your code. In light of this almost unavoidable event, what will you do to prevent, detect and recover from such errors?
  We will first run and test our codes locally, trying to design many test cases to fully test our codes. Also, we will commit frequently, so that we can revert to a previous version if some errors occur due to new changes.

Question 5: Git and Code Review

In this project, we will set up a GitHub repository for the team. We expect you to follow best practices in your development effort. We have a primer on the best practices of using GitHub, please go through it carefully.

Teams are required to conduct code reviews for every piece of code written by a team member. This is usually done using the Pull Request feature on GitHub. Upon receiving a pull request, another member of the team is supposed to review the code and approve the pull request. This makes sure that the production code has been reviewed and works as expected. Skipping this process usually leads to many hidden bugs, wasted time, and frustrations. We require you to fully adopt this best practice.

Please provide the commands on how to do the following:

- How do you create a pull request on git?
  After making the changes in the new branch, push it to the repo, click the compare&pull request button and create pull request to open a new pull request.
- How do you create a new branch on git?

By using: git branch new_branch to create a new branch. After that we can use: git checkout new_branch to switch to that branch.

- How do you merge conflicts on git?
  We first read the code comment written by teammates. If the comment doesn't help much, we then set up a Zoom meeting to exchange thoughts on their own code.
- What's the difference between a git commit and git push?
  Git commit saves the changes only to the local repository but not to the remote repository. Git push updates your committed changes and allows you to send them to the remote repository where all of the teammates can see and access them.
- When should you push and pull? (e.g., when my feature completes, every few days, every day)
  Git pull is needed when my feature completes and needs my teammate for code review. Git push is needed a few times a day since more frequent pushes means lower conflict possibility.
- What should you look for when conducting a code review?
  I should look for what to review, for instance, which segment in codes has been changed. Then, I should write test cases to test the code to make sure that there's no explicit bugs. Finally, I should give feedback and communicate with my teammates about our goals and expectations.
- How should you report bugs found in a code review?
  I should be specific about where the bugs are and the test cases related to it. Also, it's often a good practice to have a clearly specified bug number. After that, I will communicate with my teammates about the bugs and fix it together.

Question 6: Profiling and Debugging

Debugging capabilities are going to be crucial to achieving the set targets for this project. The easiest way for debugging is logging. Logging requests will help you understand where the bottleneck might be. Logging libraries offer additional features like log levels, log line number, timestamp, etc. So it's advisable to use a logging library. **Caution: logging in production systems can lead to a drop in performance, so only use logging while developing and debugging.**

- What should you look for when selecting a logging library?
  We will look for a logging library with a simple, intuitive API.
- How do you monitor your VM's CPU usage, memory usage if you have direct access to them? If the VMs are in a Kubernetes cluster and you don't have SSH access into them, how would you monitor their CPU and memory usage? How do you monitor the CPU and memory usage of a Kubernetes pod? Give specific commands. How will you identify if one of them is the bottleneck of your system?
  If we have direct access to VMs, we can ssh into the VMs and use commands to monitor the CPU usage. If we don't have SSH access, we can use some profiling tools like VisualVM to see the CPU usage and memory usage. To monitor the CPU usage of a pod, we can go to "cd /sys/fs/cgroup/cpu" and run "cat cpuacct.usage". To monitor the memory usage of a pod, we can go to "cd /sys/fs/cgroup/memory" and run "cat memory.usage_in_bytes". We can use

profiling methods introduced in the Profiling a Cloud Service primer to identify the bottleneck of our system.

- How can you find out the execution details of a MySQL query? (e.g., what indexes it will use etc.)
  Use EXPLAIN statement.
- What is the time complexity of an HBase GET operation? (Hint: it should be related to the number of HFiles.)
  O(logN)
- Read through the MySQL primer carefully. Quote the terminologies that you think will impact the database tier performance.
  Indexing, join, indexing design.
- Read through the HBase primer carefully. Quote the terminologies that you think will impact the database tier performance.
  Scan, get, rowkey design.

Question 7: Integration

The integration between the web-tier and storage-tier will play a crucial role in getting good performance in Microservice 3. Integration is typically achieved through database libraries. There are multiple libraries for doing integration with MySQL or HBase. The performance can vary between libraries and configurations so it will behoove the team to experiment with different libraries and configurations.

- What features would be crucial for achieving the target throughput? (e.g., connection pooling, prepared statements)
  Cache, connection pooling and prepared statements.
- Name a few libraries that you might consider?
  Java can use JDBC connection pools and prepared statements, or Vert.x database clients. The C# service uses MySqlConnector: each request obtains a pooled connection, executes parameterized SQL and reads results asynchronously. Request data stays local to the handler.

5% Bonus of Microservice 1

Fill in the form if your team was able to achieve the Microservice 1 target by 2/27 and are hence eligible for the 5% early bird bonus. Please push all the files you wrote to reach the target RPS and tag your git commit as “earlyBird-m1” so that we know you completed this bonus at that commit.

	Microservice 1
score	
submission id	

throughput	
latency	
correctness	
error	
date/time	